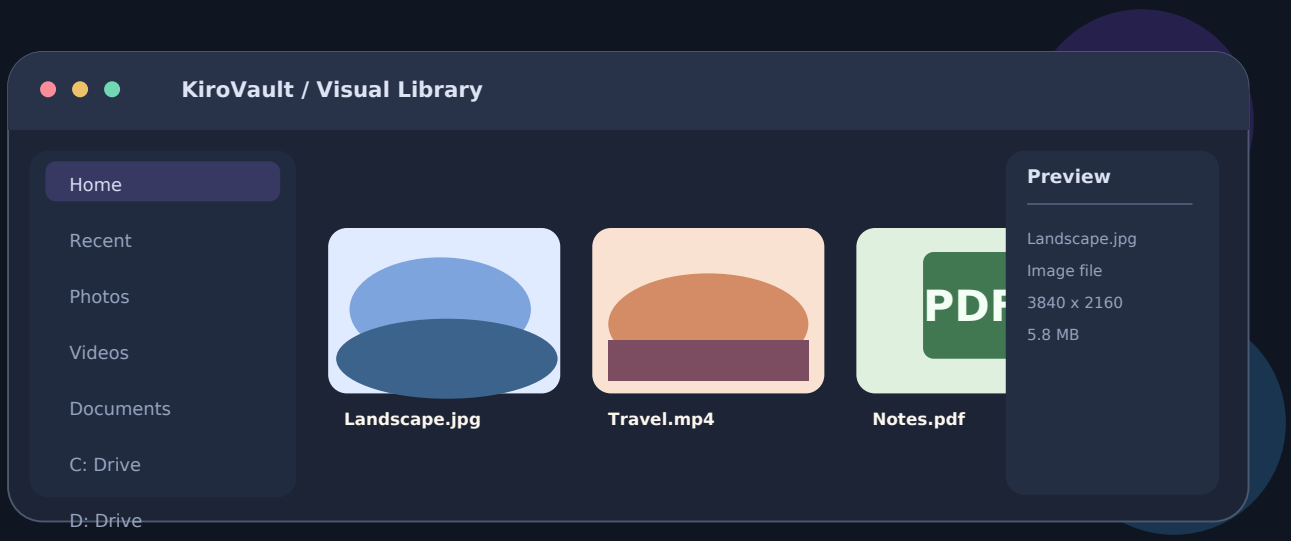
PRODUCT SPECIFICATION / WINDOWS DESKTOP

KiroVault

A visual file manager for the entire PC.

Premium glass-inspired design. Real folders and files. Rich previews. Smart local organization.



MASTER BLUEPRINT | Product vision, complete feature catalogue, UI layouts, preview matrix, project structure, architecture, data model, implementation roadmap, quality gates, and source references.

Final app name: KiroVault
Platform: Windows desktop, local-first for v1
Document version: 1.0 | 23 October 2026

DOCUMENT MAP

Contents

Use this document as the shared product and engineering brief for the first KiroVault build. External source names throughout the PDF are clickable.

Contents . . . 2

Vision and positioning . . . 5

Brand usage . . . 5

Core principles . . . 5

Initial users and primary jobs . . . 5

Scope, boundaries, and non-goals . . . 6

Version 1 scope . . . 6

Later enhancements . . . 6

Non-goals for the initial release . . . 6

Glass design . . . 7

Visual references for the design language . . . 7

Category palette (starting point) . . . 7

Surface hierarchy . . . 7

Opacity and accessibility . . . 7

Primary Explorer layout . . . 8

Layout zones . . . 8

Core navigation model . . . 8

Core screens and their contents . . . 9

Screen 1 - Home dashboard . . . 9

Screen 2 - Explorer . . . 9

Screen 3 - Visual Library / Timeline . . . 9

Screen 4 - Storage . . . 9

Screen 5 - Settings . . . 9

Filesystem operations and interaction rules . . . 10

Critical operation states . . . 10

File preview support matrix . . . 11

Preview UX contract . . . 11

Thumbnails, video frames, documents, and cache . . . 12

Thumbnail pipeline . . . 12

Video frame policy . . . 12

Document and archive handling . . . 12

Cache policy . . . 12

Date grouping, views, collections, and search . . . 13

Supported views . . . 13

Date sources . . . 13

Search levels . . . 13

Smart collections . . . 13

Disk usage and duplicate safety . . . 14

Features . . . 14

Duplicate workflow . . . 14

Indexing scope and privacy . . . 14

Recommended application stack . . . 15

Layer responsibilities . . . 15

Why this stack . . . 15

Recommended project folder structure . . . 16

Frontend module map . . . 17

Rust command modules and local database . . . 18

Suggested Rust module responsibility . . . 18

Core database tables . . . 18

Database rules . . . 18

Expected behavior from click to result . . . 19

Browse a folder . . . 19

Preview a video . . . 19

Run a PC-wide scan . . . 19

Delete a duplicate candidate . . . 19

Shared UI states for every screen . . . 19

Performance, reliability, security, privacy . . . 20

Performance strategy . . . 20

Initial engineering targets . . . 20
Security and privacy . . . 20
Implementation roadmap . . . 21
Milestone priority . . . 21
Acceptance checklist . . . 22
Suggested test corpus . . . 22
Scaffold commands and first milestone . . . 23
Recommended folder path . . . 23
PowerShell scaffold . . . 23
Project development rules . . . 23
External design links and inspiration . . . 24
Official documents used in this blueprint . . . 25
The product in one implementation-ready page . . . 26
Required MVP . . . 26
Follow-on scope . . . 26
Definition of done . . . 26

How to read this blueprint

The feature catalogue is the intended product scope. The technical stack and repository tree are proposed implementation choices, not code that already exists. Visual mockups are original schematic illustrations. External inspiration references open the original source pages.

01 / PRODUCT DEFINITION

Vision and positioning

KiroVault is a premium Windows desktop file manager that makes file browsing visual without sacrificing the reliability and familiarity of real filesystem operations.

One-line product definition. KiroVault combines a glossy, iPhone-inspired visual interface with a real Windows file manager, a modular preview engine, date-based organization, and a searchable local index.

Product promise. Users should be able to open a folder, recognize photos and videos by actual content thumbnails, preview common files in place, and organize content by dates or categories without being forced to move the originals.

Brand usage

Item	Decision
Final name	KiroVault - use this in the installer, About screen, app title, documentation, and repository.
Brand name	KiroVault - use consistently as the app name across the interface and documentation.
Tagline candidate	Your files. Your world. Beautifully organized.
Visual identity	Refined glass panels, pastel file cards, layered depth, subtle highlights, and clear typography.
Primary differentiator	The visual-preview experience: content-aware file cards plus a contextual inspector.

Final-name note: KiroVault is the selected app name. Before public release, check trademark databases, domains, app stores, and relevant company-name registries. This document does not constitute trademark clearance.

Core principles

- Visual first: recognizable content thumbnails and previews should replace generic icons where supported.
- Real filesystem: folders and files remain in their ordinary Windows locations; the app indexes rather than imports the whole PC.
- Local first: ordinary browsing and preview should not require a cloud service or account.
- Fast by design: lazy loading, thumbnail caching, virtualized lists, and background work are architectural requirements.
- Safe by default: no script execution on preview, no silent destructive cleanup, and no bypassing OS permissions.
- Transparent capability: supported, partial, unavailable, and failed previews must be distinguishable.

Initial users and primary jobs

- People with large mixed collections of photos, videos, documents, downloads, and project files.
- Developers who want quick previews of source code, configuration, Markdown, JSON, and logs without opening an editor for every file.
- Users who want a polished visual layer on top of ordinary C:, D:, and external-drive storage.
- People who want to find large files or duplicate candidates without automatically deleting them.

02 / PRODUCT SCOPE

Scope, boundaries, and non-goals

Treat file management, previews, and local indexing as the core. Add more advanced intelligence only after those workflows are stable.

Version 1 scope

- Windows desktop application with proper installer and app icon.
- Drive and folder navigation across accessible locations selected by the user.
- Safe basic operations: create, rename, copy, move, delete, multi-select, drag-and-drop, context actions, and path copy.
- Real image thumbnails, video frame thumbnails and playback, PDF display, text/code preview, and a reliable unsupported-file inspector.
- Gallery, grid, list, sorting, filtering, Favorites, Recent, and basic date grouping.
- Thumbnail cache and local metadata index; global indexing should be configurable and resumable.
- Light and dark themes with category color surfaces and a responsive preview inspector.

Later enhancements

- Richer Office previews, extra codecs and RAW/HEIC formats, archive preview beyond ZIP, audio waveform generation, slideshow, duplicate hashing, content indexing, saved searches, and deeper Windows context-menu integration.
- Optional smart classification or local AI features only after privacy boundaries, controls, and a non-AI baseline have been validated.

Non-goals for the initial release

- Not a cloud drive or file-sync product. No online account is required for core tasks.
- Not a macOS Finder clone. Inspiration may come from iOS and Fluent design, but the layout and interaction should suit Windows desktop use.
- Not an operating-system cleaner that deletes files automatically.
- Not a promise to render every proprietary, encrypted, corrupted, or rare file type.
- Not a file importer. The application should not duplicate every source file into an app-managed library.
- Not an execution environment: selecting or previewing a script must never run it.

> Key product rule
> Date grouping and smart collections are virtual views. A file may appear in Photos, Videos, October 2026, and Large Files without being copied or physically moved. Moving a file is always an explicit user action.

03 / VISUAL DESIGN SYSTEM

KiroVault Glass

Use glass effects on navigation and transient controls, then keep file content surfaces calm enough for thumbnails, filenames, and metadata to remain readable.

Visual references for the design language

Apple describes materials as a way to establish depth and hierarchy, and advises using its most translucent effects sparingly. Microsoft Fluent 2 describes Solid, Acrylic, Mica, and Smoke as distinct materials with different jobs. KiroVault should borrow these principles conceptually rather than copy platform-specific controls. See Apple materials guidance and Fluent 2 materials .



Category palette (starting point)

Category	Light surface	Foreground	Use
Photos	#E1EBFF / ice blue	#294E8C	Photo cards, image filters, photo collection markers.
Videos	#F9E2D2 / peach	#80451F	Video cards, video-duration badges, playback-related elements.
Documents	#E0F0DE / mint	#2C5C39	PDF and office document cards.
Code and projects	#EDE2FF / lavender	#573783	Source code, project files, configuration, development tools.
Audio	#F9E3EC / rose	#85334F	Audio files, music collection surfaces, waveform markers.
Archives	#F4EED5 / champagne	#6C5923	ZIP and other archive containers.

Surface hierarchy

- Layer 0 - app background: neutral light or dark surface with only a faint ambient tint.
- Layer 1 - navigation: translucent/acrylic-style panel, light blur, thin border; text remains high contrast.
- Layer 2 - file content: category pastel surface or thumbnail image; avoid heavy blur behind metadata.
- Layer 3 - selection and hover: clear selection outline, subtle elevation, small highlight; never rely on color alone.
- Layer 4 - dialogs and menus: stronger opaque or frosted material for readable actions and safe confirmation.
- Typography: filename is primary, extension/type and date are secondary, path and technical metadata are tertiary.
- Motion: short, restrained easing; all major motion should respect a reduced-motion setting.

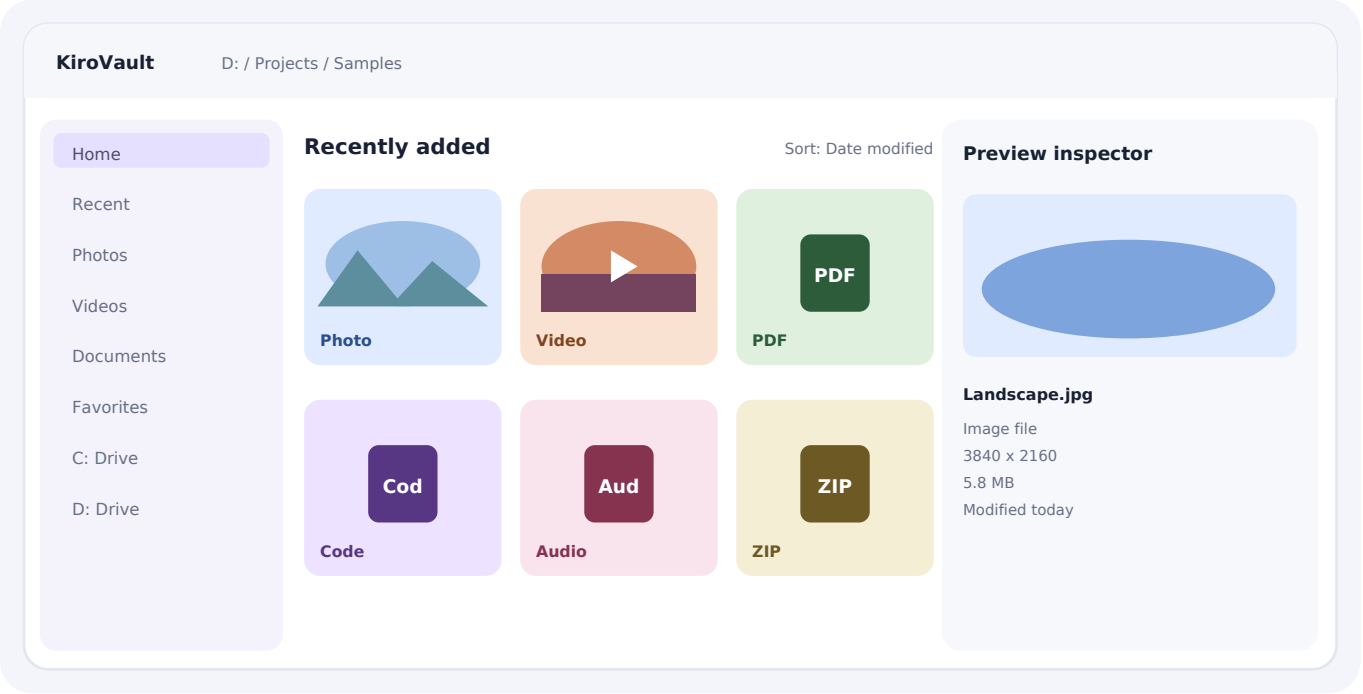
Opacity and accessibility

Use 30-50% alpha for decorative tint layers only when text is not placed directly over the transparent color. Text, icons, and controls need sufficient contrast against both light and dark backdrops. Offer a reduced-transparency or high-contrast mode. Design should be tested with real thumbnail photos that vary in brightness, not only with flat color samples.

04 / EXPERIENCE BLUEPRINT

Primary Explorer layout

The main Explorer should be a responsive three-zone workspace: persistent navigation, file content, and an optional inspector. This is an original schematic, not a screenshot of a finished app.



Layout zones

Zone	Responsibilities	Rules
App chrome	Window title, global search, navigation history, breadcrumbs, settings, window controls.	Keep persistent controls consistent across screens. Use native window behaviors where practical.
Sidebar	Home, Recent, Photos, Videos, Documents, Downloads, Favorites, drives, pinned folders, collections.	Collapsible and configurable. Drive access failures must be visible rather than silently hidden.
Main content	Grid, gallery, list, timeline, sorting, grouping, filters, multi-selection, empty/loading states.	Virtualize long lists, progressively load thumbnails, and avoid layout shifts as previews arrive.
Inspector	Preview, properties, metadata, primary actions, path, tags, favorite state.	Resizable and hideable. On narrower window widths it may collapse automatically.

Core navigation model

- Home is an overview, not a replacement for direct filesystem browsing.
- Explorer navigation must support back, forward, up, refresh, breadcrumbs, and a copyable full path.
- Gallery is a view over files, not a second storage system. It can filter to images/videos and group by capture or filesystem date.
- Search results must show the original location and allow opening the containing folder.
- Storage insights and duplicate candidates should link back to the real file location.

05 / SCREEN SPECIFICATIONS

Core screens and their contents

Each screen should reuse the same design tokens and shared components. Keep the first release focused on five coherent workspaces.

Screen 1 - Home dashboard

- Recent files and recently modified files, each with preview thumbnails when available.
- Quick access to Desktop, Downloads, Documents, Pictures, Videos, Music, Favorites, and pinned folders.
- Drive cards with capacity and used-space information when available.
- Quick collections: Photos, Videos, Screenshots, Archives, Large Files, and Recent.
- Optional widgets that users can hide or reorder; do not crowd the screen with statistics.

Screen 2 - Explorer

- Breadcrumbs and path input, search, filter chips, sort/group controls, and view selector.
- Grid/gallery/list modes with keyboard selection and bulk operations.
- Context menu for open, open with, rename, copy, cut, paste, move, delete, copy path, details, and show in folder.
- Optional inspector and persistent sort/view preference per folder.
- Accessible loading, empty, permission error, offline-drive, and unsupported preview states.

Screen 3 - Visual Library / Timeline

- Group media by Today, Yesterday, week, month, or year, with collapsible headings.
- Prefer EXIF capture time for photos and embedded media dates when reliable; otherwise label the filesystem timestamp used.
- Filters for image/video/audio, file size, dimensions, extension, and date range when metadata exists.
- Full-screen image viewer; video preview with controls; previous/next navigation.
- No physical file moves when changing timeline groupings.

Screen 4 - Storage

- Drive capacity, folder-size ranking, large-file list, type/extension breakdown, and duplicate candidates.
- Scanning should be cancellable, show progress, and explain exclusions or inaccessible folders.
- Duplicates are candidates until reviewed. Offer compare, reveal location, and explicit delete/move actions.
- Do not scan removable drives continuously after disconnection; mark their index as stale and re-check on reconnection.

Screen 5 - Settings

- Appearance: light/dark/system, accent, category colors, card density, transparency, reduced motion.
- Explorer: default view, file extension visibility, hidden files, double-click behavior, default preview pane.
- Library: indexed locations, excluded paths/extensions, scan schedule/trigger, pause/resume/rebuild, cache controls.
- Privacy and performance: local cache location, optional metadata collection controls, background work limits.
- About: app version, licenses/third-party components, diagnostic export, and update behavior if introduced.

06 / FILE MANAGEMENT

Filesystem operations and interaction rules

KiroVault must be a functional file manager. Pretty cards are not enough: operations must be correct, reversible where possible, and clear about failure.

Action	Required behavior	Safety / UX rule
Create	Create folder and supported new-file templates if implemented.	Validate names and show path. Handle invalid characters and name collisions.
Rename	Rename selected file/folder.	Detect collisions; preserve extensions by default but allow editing them deliberately.
Copy / cut / paste	Copy or move files and folders within and across drives.	Show progress for large operations; handle conflicts with Skip, Replace, Keep both, or Cancel.
Move	Move to another path or via drag-and-drop.	Use a clear drop target and show final destination before ambiguous moves.
Delete	Move to Recycle Bin when available; permanent deletion only when explicitly selected.	Confirm multi-file/permanent deletes and report partial failures.
Open	Open with the Windows-associated application.	Do not execute files from preview or on selection. Open is a separate intentional action.
Context menu	Open, Open with, Rename, Copy path, Move, Copy, Delete, Properties, Reveal in folder.	Actions must be enabled/disabled based on current selection and permissions.
Multi-select	Ctrl/Shift selection, keyboard navigation, bulk operations.	Display selection count and avoid accidental actions on a large selection.
Drag-and-drop	Move or copy files/folders with clear feedback.	Differentiate within-app reorder from real filesystem drag/drop. Ask when the operation is ambiguous.
Hidden files	Optional show-hidden toggle.	Avoid surfacing protected system files by default; obey permissions.

Critical operation states

- Idle: action ready and enabled.
- Running: progress indicator and cancellation where supported.
- Conflict: explicit choice for existing names, duplicates, or changed destination.
- Partial failure: list completed and failed items; do not hide failures.
- Success: concise confirmation with Undo where the operation can be reversed safely.
- Recovery: if interrupted, reconcile the actual filesystem state before resuming a queued job.

> Path safety
> Validate path arguments on the Rust side, not only in the frontend. Canonicalize paths where appropriate, constrain operations to intended scopes, handle junctions/symlinks carefully, and never treat a UI string as proof that access is authorized.

07 / PREVIEW ENGINE

File preview support matrix

Use a modular preview router so each file type has its own renderer. The inspector must always have a fallback instead of turning into a blank panel.

Type / examples	V1 preview behavior	Potential implementation	Fallback / caveat
Images: JPG, PNG, GIF, BMP, WebP	Real thumbnails, zoom, fit, full screen, dimensions.	Native Windows Shell thumbnails or image decoder; metadata extractor.	GIF animation may be a first frame only until animation support is added.
AVIF, HEIC, RAW	Best-effort thumbnail and metadata support.	Format-specific decoder/plugins.	Mark format compatibility as conditional; do not claim universal support.
Video: MP4, MOV, MKV, WebM, AVI	Frame near start, duration/resolution, embedded playback, seek bar.	FFmpeg/ffprobe or compatible platform decoder; cached preview frames.	Container and codec are separate. Some files may need a bundled decoder not present by default.
PDF	Page 1 thumbnail, page navigation, text/search if supported.	PDF.js or a dedicated native rendering integration.	Encrypted or damaged files display a clear error.
DOCX, XLSX, PPTX	Read-only document/slide/sheet preview when supported.	Dedicated Office renderer or controlled conversion pipeline.	Complex layouts, charts, macros, embedded objects, and fonts may differ from Office.
Source code	Syntax-highlighted read-only preview and line numbers.	Text reader + syntax-highlighting component.	Limit maximum preview bytes/lines for huge files.
Text / Markdown / JSON / XML / YAML / logs	Readable text, optional syntax highlighting, load-more for large files.	Text reader with encoding detection and safe rendering.	Escape HTML; never render a file as trusted active web content.
Audio: MP3, WAV, FLAC, AAC, etc.	Play/pause, duration, metadata; waveform later.	Platform audio APIs or compatible media engine.	Format/codec availability varies; waveform processing should be asynchronous.
ZIP and archives	List members and sizes without extracting.	Archive library with path traversal defenses.	Do not extract by default; block unsafe member paths if extraction is added.
Fonts	Font family/style/sample preview where supported.	Read-only font parser/OS information.	Never install fonts silently.
Unknown / proprietary	Icon, extension, path, size, timestamps, open-with.	Generic metadata provider.	Show “Preview unavailable” with a useful reason when possible.

Preview UX contract

- Single select: update inspector; do not launch external apps.
- Double-click / Enter: open with associated application by default.
- Video hover: optional muted loop only when enabled; provide a performance setting.
- Thumbnail source: use the actual file contents where possible, not generic category art.
- Progressive loading: show skeleton/placeholder, then replace with cached or new thumbnail.
- Failure: show a stable fallback card, error reason, and retry/open action.
- Security: preview read-only; no macros or scripts executed; remote content blocked by default.

08 / PREVIEW PIPELINE DETAILS

Thumbnails, video frames, documents, and cache

The preview engine should be demand-driven. A folder containing thousands of files must not cause every full file to be decoded at once.

Thumbnail pipeline

- 1. File listing returns canonical path, file name, extension, size, dates, and a thumbnail-state key.
- 2. UI requests a thumbnail only for visible or near-visible cards; virtualized lists reduce requests.
- 3. Cache lookup uses path plus change markers such as size and modification time, with an optional content hash for high-confidence invalidation.
- 4. On a cache miss, dispatch a bounded background job to the correct thumbnail provider.
- 5. Write the result to a local cache, then update the UI without forcing a full list rerender.
- 6. Invalidate cache entries when file path, file size, modification time, or chosen decoder/version changes.

Video frame policy

- Default thumbnail frame: near the start, but not necessarily the first frame if it is black or blank. Make this deterministic and bounded.
- Do not generate full-resolution poster images for every video. Generate thumbnail-size frames and optionally a higher-resolution inspector poster on demand.
- Read duration, dimensions, frame rate, codec/container information when available.
- Playback should support a seek bar, current time, total duration, mute/volume, and pause/play.
- Separate container support from codec support; an MP4 can still contain a codec the chosen decoder cannot play.
- Bundle and distribute media components only after evaluating license, patent, build configuration, and attribution requirements.

Document and archive handling

- PDF: load first page quickly, then render later pages on demand; avoid rendering every page when only one file card is visible.
- Office: prefer a well-tested renderer. If converting for preview, create a cache artifact and never overwrite the original file.
- Text/code: cap the initial read size, detect encoding safely, escape active markup, and provide an explicit “load more” action.
- ZIP: list names, sizes, and nested structure without extracting. If extraction is later supported, defend against path traversal and unexpected overwrite.
- All preview workers should enforce timeouts and per-file resource limits.

Cache policy

Cache content	Rule
Thumbnails	Keep separate from original files; maintain metadata and rebuild invalid entries safely.
PDF preview render	Generate first pages on demand and purge by size/age budget.
Video posters	Store multiple sizes only if useful; no full video duplication.
Text preview	Prefer transient memory cache; avoid permanently storing sensitive document contents unless explicitly required.
Cache controls	Settings should allow clearing cache and show its disk footprint. Clearing cache must not delete source files.

09 / SMART ORGANIZATION

Date grouping, views, collections, and search

Virtual organization gives users more ways to discover the same underlying files without disrupting folder structure.

Supported views

View	Best for	Required behavior
Grid	Mixed folders and general browsing.	Adjustable card size, file-type tints, name + compact metadata.
Gallery	Photos and videos.	Larger thumbnails, aspect-ratio-aware cells, selection and quick preview.
List / details	Dense folders and power users.	Sortable columns for name, type, size, created, modified, and path.
Timeline	Media and recently changed items.	Group by relative day/week/month/year; collapsible headers and date filters.
Storage view	Space usage.	Folder-size ranking, categories, large files, duplicate candidates.

Date sources

- Use photo capture metadata when available and valid; label it as “Date taken” rather than silently replacing filesystem dates.
- Show creation and modification timestamps as separate fields.
- For videos/audio, use embedded media metadata only where reliable; filesystem modification date is a safe fallback.
- Make the active grouping rule visible to users and let them switch the rule without moving files.
- Timezone handling should be consistent and should not unexpectedly shift older dates during grouping.

Search levels

Level	Fields	Notes
Immediate folder search	Name, extension, visible folder scope.	Works even before a whole-drive scan finishes.
Indexed local search	Name, path, type, size, dates, metadata.	Use SQLite indexes; update incrementally.
Content search (later)	Text extracted from supported PDFs and Office files.	Opt-in/configurable because content indexing increases scan time and sensitivity.
Advanced filters	Location, type, size range, date range, favorite, tag, duplicate candidate.	Combine filters predictably; offer saved searches later.

Smart collections

- Recent, Recently Modified, Photos, Videos, Audio, Documents, Archives, Downloads, Screenshots, Large Files, Duplicate Candidates.
- User-defined collections from a saved search or chosen paths.
- A file may appear in multiple collections without duplicates on disk.
- Make the real path available from every collection result.

10 / STORAGE AND FILE INSIGHTS

Disk usage and duplicate safety

Storage analysis should help the user make informed choices, never silently clean or delete files.

Features

- Drive usage: total, used, free, and percentage when available.
- Largest files, largest folders, extension/type distribution, and items not modified for a user-selected period.
- Duplicate candidates by exact size/hash and, optionally, near-duplicate image analysis in a later phase.
- Folder-size calculation with scan progress, cancellation, and notes where inaccessible children were skipped.
- Removable drive state: mark results as stale after disconnect; revalidate when connected again.
- Quick navigation from every insight result to the source path.

Duplicate workflow

Identify candidates -> show file names, locations, sizes, and comparison details -> user reviews -> user explicitly chooses keep/delete/move -> operation result is logged. Do not delete by default. A visual similarity match alone is insufficient proof of a duplicate.

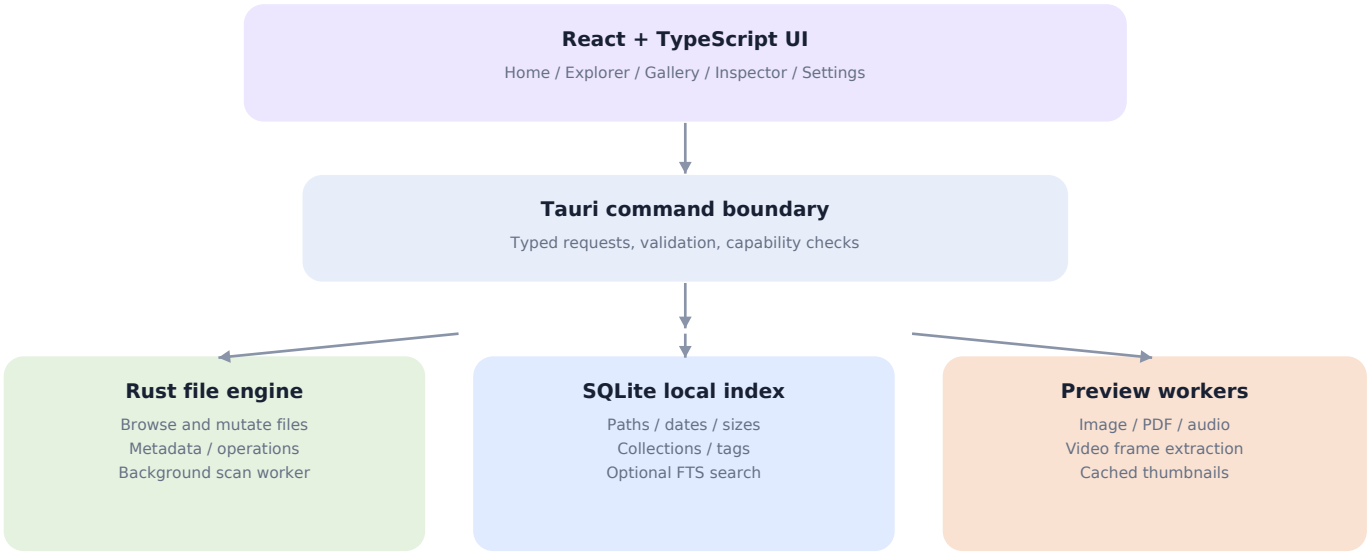
Indexing scope and privacy

- Default to common user locations or user-selected drives; do not assume every OS/system path should be indexed.
- Allow exclusions by path, folder, extension, or size.
- Pause, resume, cancel, and rebuild index; show last scan time and issues.
- Store index and cache in an app-specific local directory, not beside source files unless explicitly chosen.
- Make the database rebuildable; deleting it must not delete the originals.
- Avoid remote uploads, telemetry of file names, and indexing file contents without clear settings and consent.

11 / TECHNICAL ARCHITECTURE

Recommended application stack

Tauri 2 + React + TypeScript + Rust + SQLite is the recommended starting stack for a Windows-first desktop application. Validate package support and licensing for every preview dependency before bundling.



Local-first: original files remain in their real locations; cache and index are rebuildable.

Layer responsibilities

Layer	Recommended technology	Responsibility
Desktop shell	Tauri 2	Window lifecycle, app packaging, controlled frontend-to-native command boundary.
Frontend	React + TypeScript	Pages, UI state, accessible interactions, grid/list/timeline, selection and preview.
Build tool	Vite	Local development server and optimized frontend build.
Native engine	Rust	Directory listing, file operations, background workers, metadata extraction, safe path validation.
Local index	SQLite	File index, collections, tags, user settings, search indexes.
Full-text search (optional)	SQLite FTS5	Search extracted text in supported documents if content indexing is enabled.
Video metadata/posters	FFmpeg/ffprobe or Windows/media-specific integrations	Inspect media, extract selected frames, support controlled preview workers.
PDF renderer	PDF.js or native integration	Page rendering and page navigation, subject to library packaging/licensing.
Windows thumbnail fallback	Shell thumbnail interfaces	Use Windows-provided thumbnails/icons where available.

Why this stack

- React/TypeScript provide a maintainable UI component system and typed state/API boundaries.
- Rust is a good place for filesystem work, background jobs, and native performance-sensitive tasks.
- Tauri can package the application with a web-based frontend, but must be configured with narrow permissions and carefully validated commands.
- SQLite can persist the index locally. Full-text search is an optional extension, not a requirement for the first folder browser.
- Native Windows thumbnail interfaces may reuse system handlers, while a modular renderer still fills format gaps.

Primary stack references: Tauri project creation, React + TypeScript, Vite guide, SQLite FTS5, FFmpeg documentation, PDF.js.

12 / REPOSITORY BLUEPRINT

Recommended project folder structure

This is a proposed scalable structure for the first implementation. Keep frontend views separate from reusable components, filesystem services, native Rust commands, and tests.

```
kirovault/
|-- package.json
|-- package-lock.json
|-- vite.config.ts
|-- tsconfig.json
|-- index.html
|-- README.md
|-- public/
|   |-- brand/
|   |-- icons/
|   `-- placeholders/
|-- src/
|   |-- app/
|   |   |-- App.tsx
|   |   |-- router.tsx
|   |   `-- providers/
|   |-- components/
|   |   |-- shell/
|   |   |-- files/
|   |   |-- preview/
|   |   `-- ui/
|   |-- features/
|   |   |-- home/
|   |   |-- explorer/
|   |   |-- gallery/
|   |   |-- timeline/
|   |   |-- search/
|   |   |-- storage/
|   |   |-- favorites/
|   |   `-- settings/
|   |-- hooks/
|   |-- services/
|   |-- styles/
|   |-- types/
|   `-- utils/
|-- src-tauri/
|   |-- Cargo.toml
|   |-- tauri.conf.json
|   |-- capabilities/
|   |   `-- default.json
|   `-- src/
|       |-- main.rs
|       |-- lib.rs
|       |-- commands/
|       |-- db/
|       |-- models/
|       `-- services/
|-- tests/
|   |-- unit/
|   |-- integration/
|   `-- fixtures/
`-- docs/
    |-- architecture.md
    |-- preview-support.md
    |-- privacy-model.md
    `-- release-checklist.md
```

Frontend module map

Path	Contents
src/app/	Root app, routes, providers, global error boundaries.
src/components/shell/	AppSidebar, TopBar, breadcrumbs, command bar, PreviewInspector, responsive layout.
src/components/files/	FileCard, FolderCard, FileGrid, FileList, FileIcon, context menu, selection toolbar.
src/components/preview/	PreviewRouter and one renderer per supported format family.
src/features/	Page-level logic for Home, Explorer, Gallery, Timeline, Search, Storage, Favorites, Settings.
src/services/	Typed wrappers around native commands: filesystem, indexing, thumbnails, metadata, search.
src/styles/	Design tokens, light/dark themes, category colors, glass materials, motion and accessibility settings.
src/types/	Shared file records, preview states, sort/group models, command request/response types.

13 / NATIVE ENGINE AND DATA

Rust command modules and local database

Use a typed command boundary. Frontend code requests operations; Rust validates them and returns structured results. Do not expose unrestricted raw filesystem access to every UI component.

Suggested Rust module responsibility

Module	Responsibility
commands/filesystem.rs	List directory, stat, create, rename, copy, move, delete, open-with, reveal-in-folder, path copy support.
commands/indexing.rs	Start/pause/resume/cancel scans, return progress, update records, report skipped paths.
commands/thumbnails.rs	Request/cancel thumbnails, cache lookup and invalidation, bounded worker queue.
commands/metadata.rs	Read supported image/audio/video metadata using safe, bounded readers.
commands/search.rs	Search by name/path/type/dates/size; later route supported content queries to FTS.
db/	Connection lifecycle, schema migrations, prepared queries, indexes, integrity checks.
models/	Serializable structs for FileEntry, MediaMetadata, PreviewStatus, ScanProgress, SearchFilter.
services/	Platform adapters, thumbnail providers, preview worker coordination, filesystem watcher if added.

Core database tables

Table	Main fields / role
files	id, canonical_path, parent_path, name, extension, kind, size_bytes, created_at, modified_at, indexed_at, source_root, scan_status.
media_metadata	file_id, width, height, duration_ms, codec/container hints, capture_time, audio metadata, metadata_source.
thumbnail_cache	cache_key, file_id/path key, thumbnail_path/blob reference, width, height, generator_version, source_fingerprint, created_at.
collections	id, name, type, query_json/rule, created_at; for saved queries or curated path collections.
favorites	id, path, item_kind, pinned_at.
tags	id, name, color.
file_tags	file_id/path key, tag_id; allow many-to-many tags.
settings	key, value_json, updated_at.
scan_jobs	id, root_path, status, cursor/checkpoint, counters, errors, started_at, updated_at.
optional content_fts	FTS5 index for extracted text from supported formats only, subject to explicit scope and user settings.

Database rules

- Persist canonical paths and revalidate them when a result is opened. A record can become stale because a file was moved or deleted outside KiroVault.
- Use schema versioning and migrations; never assume the local index is always valid.
- Use prepared statements and parameter binding.
- If SQLite WAL is selected for concurrency, use a maintained SQLite build and follow the official caveats. WAL is not appropriate for a shared network-mounted index database.
- Keep source files, thumbnails, and index clearly separated; the index/cache must be rebuildable.

14 / KEY USER FLOWS

Expected behavior from click to result

These are the baseline interaction contracts. The exact visual animation may evolve, but behavior and safety rules should remain stable.

Browse a folder

- User clicks drive or folder -> native command validates access -> directory names and metadata return -> UI renders visible rows -> visible thumbnails are requested asynchronously.
- If a folder is inaccessible, show a permission error and preserve the ability to navigate back. Do not hang the whole app.
- If the drive disconnects, retain UI state where useful, mark records unavailable, and prompt to reconnect rather than silently changing paths.

Preview a video

- User selects video -> inspector appears -> metadata/cache lookup -> poster thumbnail displayed or loading placeholder shown.
- User presses play -> supported playback engine loads file and controls initialize.
- If codec/container is unsupported, show the detected details if available, explain playback is unavailable, and offer Open with external app.
- Hover previews are optional and must not autoplay with sound.

Run a PC-wide scan

- User selects scan roots/exclusions -> summary shows estimated scope if feasible -> scan starts in background.
- Progress shows roots visited, items indexed, errors/skips, pause/resume/cancel.
- Folder browsing remains available while indexing continues.
- On app restart, either resume a safe checkpoint or mark the prior job interrupted and offer resume/rebuild.
- The app never requests admin privileges automatically just to bypass a permission failure.

Delete a duplicate candidate

- Candidate list contains hash/size evidence and file locations -> user compares -> chooses the specific item to remove.
- Show whether deletion will use Recycle Bin or be permanent.
- Perform the action -> report success or partial failure -> refresh index and invalidate thumbnails.
- Never delete automatically based only on file name, size, or visual similarity.

Shared UI states for every screen

State	Required response
Loading	Skeleton/placeholder, non-blocking progress, no layout collapse.
Empty	Explain why there are no results and give a useful next action.
Permission denied	Name the blocked location, explain access is restricted, and provide a route back.
Drive unavailable	Mark stale/offline, keep other locations functional, re-check on reconnect.
Preview unsupported	Show metadata, open-with action, and a plain reason rather than a blank panel.
Operation conflict	Offer explicit choices and state which item would be affected.
Operation partially failed	Provide per-item outcome and do not falsely report complete success.

15 / QUALITY ATTRIBUTES

Performance, reliability, security, privacy

Quality requirements should guide implementation from day one. Numbers below are targets to test, not measured claims about an existing application.

Performance strategy

- Virtualized lists: render visible items and a small overscan region, not every row in a huge directory.
- Lazy thumbnails: request visible/near-visible cards, prioritize selected item, cap concurrency, and cancel requests when folders change.
- Cache: persist generated previews and reuse them when source fingerprints still match.
- Worker queues: separate filesystem scan, metadata extraction, hashing, document render, and video frame work from the UI thread.
- Bounded memory: avoid whole-file reads and full-resolution decoding for grid cards.
- Search: local folder search is available immediately; PC-wide search depends on the configured index state.
- Telemetry: no default collection of filenames, paths, file contents, or library activity.

Initial engineering targets

Area	Target / test
Launch	Window becomes interactive before any full-drive scan completes.
Folder open	Show names/metadata as soon as returned; never wait for all thumbnails.
Large folders	Remain responsive with several thousand entries through virtualization and bounded thumbnail work.
Scan behavior	Pause/cancel works; UI browsing continues during scanning.
Cache	Repeated folder visits should visibly reduce thumbnail-generation work.
Recovery	Interrupted scans and stale paths do not corrupt the index or delete originals.
Failure handling	Permission issues, unsupported formats, damaged files, and disconnected drives produce understandable states.

Security and privacy

- Use Tauri capability/permission allowlists narrowly. Prefer scoped commands over blanket filesystem permissions.
- Treat file paths, filenames, metadata, and document contents as untrusted input.
- Render text previews as escaped text; sanitize/disable active HTML and remote resource loads.
- Do not execute scripts or Office macros; do not silently install fonts or extract archives.
- Validate paths and symlink/junction behavior in Rust before destructive actions.
- Use explicit confirmation for permanent deletion, bulk changes, and potentially destructive conflicts.
- Store local index/cache in app data; make their location and clear-cache behavior transparent.
- Document third-party licenses. FFmpeg packaging choices can alter license obligations; review build options and legal requirements before distributing.

References: Tauri filesystem permissions, Windows file access permissions, FFmpeg license and legal notes.

16 / DELIVERY PLAN

Implementation roadmap

Build in layers with a usable product at the end of every phase. Avoid starting global indexing, duplicate analysis, and every file format before the core Explorer is functional.

Phase	Build scope	Exit criteria
0 - Foundation	Create Tauri + React + TypeScript project; configure Windows build; establish brand tokens, routing, error boundary, typed command conventions.	Application window runs; consistent design system, repo setup, build scripts, and basic settings are in place.
1 - Real Explorer	Drive/folder browser, breadcrumbs, back/forward, grid/list, file operations, selection, context menus, path and error handling.	Can browse real files and perform operations safely without any indexing requirement.
2 - Preview MVP	Image thumbnails/viewer, video poster/playback, PDF display, text/code preview, inspector, cache, unsupported fallback.	Common files are previewed in place; cache and worker queues are bounded.
3 - Smart organization	Recent, Favorites, type collections, date/timeline view, sort/filter, first local SQLite index, scan controls.	Users can organize virtually and search configured locations while app remains usable.
4 - Storage intelligence	Folder sizes, large files, duplicate candidates, optional content search, archive/office/audio improvements.	Results are traceable to real paths, and destructive actions require explicit user confirmation.
5 - Hardening and release	Stress tests, permission/drive disconnect tests, performance profiling, accessibility, licenses, installer/uninstaller, backup/recovery notes.	Release candidate passes acceptance checklist and install/uninstall tests.

Milestone priority

The first usable milestone is Phase 1 plus a polished preview of common images and videos. Search across the entire PC is valuable, but it should not block a functioning Explorer. The second milestone adds indexing and timeline views; storage analysis follows after the index is reliable.

17 / RELEASE GATES

Acceptance checklist

Use this checklist to decide whether a KiroVault build is usable. Every item is testable and should be checked against real folders and files on the target PC.

Area	Pass criteria
Startup and UI	App opens without requiring internet; window layout adapts; light/dark mode persists; text remains readable over translucent surfaces.
Navigation	C:, D:, and selected external drives are browsable when accessible; back/forward/up/breadcrumbs work; empty and permission states are understandable.
Operations	Create, rename, copy, move, delete, multi-select, drag/drop, context menu, and copy path work; collisions and partial failures are reported.
Previews	Images show actual thumbnails; videos show a real frame and can play when compatible; PDF and text/code previews work; unsupported formats have a fallback.
Organization	Grid/list views, sorting, type grouping, and date grouping work without changing source locations.
Indexing	Scan scope/exclusions are configurable; progress/pause/cancel works; stale paths can be reconciled; browsing continues during scan.
Storage insights	Folder sizes and large files link to original paths; duplicate candidates are not deleted automatically.
Performance	Long lists are virtualized; thumbnails are lazy and cached; huge files do not lock the UI or cause uncontrolled memory use.
Security	Scripts do not execute on preview; path inputs are validated natively; filesystem scope is intentional; deletion has safe confirmation.
Release	Installer/uninstaller work; app identity/icon are consistent; third-party licenses are documented; logs avoid dumping private file content.

Suggested test corpus

- Small and large JPG/PNG/WebP files; transparent images; tall and wide aspect ratios; corrupted image.
- Short and long MP4/MOV/MKV/WebM files with different codecs; variable frame rate; missing metadata.
- PDF with many pages, encrypted PDF, damaged PDF; DOCX/XLSX/PPTX with common and complex formatting.
- Very large text/log file, UTF-8 and alternative encoding, JSON with long lines, huge source file.
- ZIP with nested members, unusual filenames, and paths that would be unsafe to extract.
- Folders with thousands of items, inaccessible paths, junctions/symlinks, disconnected USB drive, and filenames with Unicode characters.

18 / WINDOWS DEVELOPER SETUP

Scaffold commands and first milestone

These commands are a starting point for a new project directory. Confirm the current Tauri prerequisite guide for Rust, the Windows C++ build tools, and WebView2 requirements before building.

Recommended folder path

```
D:\Projects\KiroVault
```

PowerShell scaffold

```
# 1) Create the React + TypeScript frontend with Vite
mkdir D:\Projects\KiroVault
cd D:\Projects\KiroVault
npm create vite@latest . -- --template react-ts

# 2) Install the initial frontend dependencies
npm install

# 3) Add Tauri 2 to the existing frontend
npm install -D @tauri-apps/cli@latest
npx tauri init

# 4) Start the desktop app in development mode
npx tauri dev

# 5) Build a release bundle when the foundation is ready
npx tauri build
```

Before running the commands
Use the current official Tauri prerequisites guide for exact Windows requirements. The create/init prompts ask for the app name, identifier, frontend dev command, and build command. A proposed bundle identifier is com.kirovault.desktop; verify ownership and naming before public release.

Project development rules

- Pin dependencies and keep package lockfiles under version control.
- Use feature branches and create tests before destructive file operations are considered complete.
- Do not grant blanket filesystem access without a documented reason. Design and verify the intended permission boundary early.
- Maintain README, architecture notes, preview support matrix, privacy model, and release checklist in docs/.
- Review Tauri and media-library changelogs before upgrading major dependencies.

19 / VISUAL REFERENCE LIBRARY

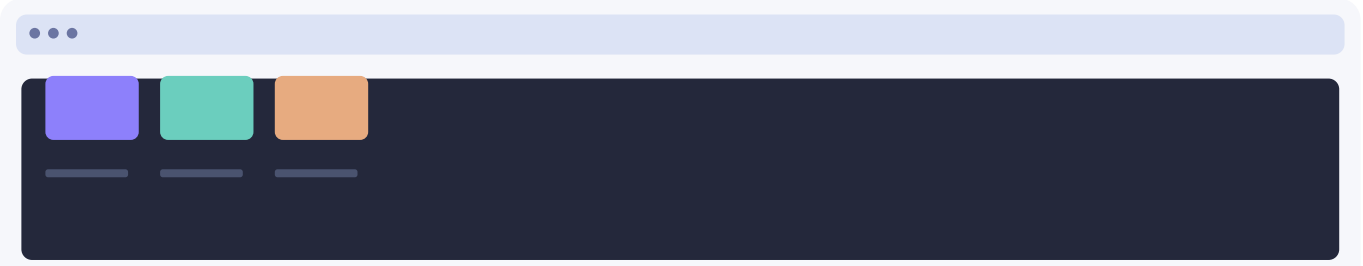
External design links and inspiration

The thumbnails below are original schematic previews drawn for this specification. Click each reference title to open the original design/source page; external pages may change or require their own terms of use.



Windows File Explorer concept

Windows File Explorer concept - Fluent-style folder layout, transparency, grouped files.



Files app - Acrylic design

Files app - Acrylic design - An actual Windows file-manager project using acrylic styling.

Windows 11 file manager UI

Windows 11 file manager UI - A community visual exploration of a blue, thumbnail-first file manager.

> Reference use
> Use these links to study layout, spacing, thumbnail treatment, materials, and Windows conventions. Do not copy source artwork or ship third-party designs/assets without checking their usage rights. KiroVault should have its own logo, component system, and original screens.

20 / TECHNICAL REFERENCES

Official documents used in this blueprint

All titles are clickable links. Because library APIs and Windows behavior can change, verify the documentation and the exact versions selected during implementation.

R1 - Tauri: Create a project
Official Tauri scaffold workflow and frontend framework support. [v2.tauri.app]

R2 - Tauri: File System plugin
Filesystem plugin capabilities, scopes, and permission model; use narrow capabilities. [v2.tauri.app]

R3 - Tauri: Prerequisites
Current environment requirements for setting up Tauri on the target platform. [v2.tauri.app]

R4 - React with TypeScript
Typing React components and TypeScript configuration basics. [react.dev]

R5 - Vite guide
Development server and project build workflow. [vite.dev]

R6 - SQLite FTS5
Optional full-text indexing for content search. [www.sqlite.org]

R7 - SQLite WAL
Concurrency trade-offs and operational notes for write-ahead logging. [www.sqlite.org]

R8 - PDF.js getting started
PDF parsing/display layers and viewer architecture. [mozilla.github.io]

R9 - FFmpeg documentation
Video/audio metadata and media processing references. [www.ffmpeg.org]

R10 - FFmpeg license/legal notes
Distribution/licensing considerations. Review the exact build configuration and dependencies. [ffmpeg.org]

R11 - Windows Shell thumbnail providers
Windows thumbnail provider behavior and cache concepts. [learn.microsoft.com]

R12 - IShellItemImageFactory
Windows API that can return icons or thumbnails for Shell items. [learn.microsoft.com]

R13 - Windows file access permissions
Permission boundaries and restricted broad file-system access guidance. [learn.microsoft.com]

R14 - Apple Human Interface Guidelines: Materials
Principles for materials, layering, translucency, and readability. [developer.apple.com]

R15 - Microsoft Fluent 2: Material
Definitions and usage guidance for Solid, Acrylic, Mica, and Smoke. [fluent2.microsoft.design]

R16 - Files app source repository
Open-source Windows file manager to study for feature and interaction references; respect its licenses. [github.com]

R17 - MDN: Media formats
Format/codec compatibility concepts; browser support differs by engine and system. [developer.mozilla.org]

Source and date note
Reviewed for this blueprint on 23 October 2026. Official documentation is preferred for implementation details. Visual design links are inspiration only and are not claims of affiliation or endorsement.

21 / FINAL BUILD BRIEF

The product in one implementation-ready page

This section can be copied into a project README or used as a handoff brief for an AI coding agent or developer.

Build KiroVault, a Windows-first desktop file manager using Tauri 2, React, TypeScript, Rust, and SQLite. Create a premium iPhone-inspired but Windows-appropriate UI using controlled glass materials, soft category colors, rounded cards, polished transitions, and strong text contrast. The app must browse real drives and folders, perform normal filesystem operations safely, and preserve the original filesystem structure.

Required MVP

- Home dashboard, Explorer, Visual Library/Timeline, Storage overview, and Settings.
- Drive/folder browsing, breadcrumbs, navigation history, search in current folder, grid/list, sorting, filtering, context menus, selection, rename/copy/move/delete/create-folder, and copy path.
- Actual image thumbnails and viewer; video frame thumbnail and in-app playback where supported; PDF and read-only text/code preview; unsupported preview fallback.
- Optional inspector, category color surfaces, light/dark theme, favorites, recent files, and basic date grouping.
- Local thumbnail cache, safe metadata model, background work queue, and clear permission/error states.

Follow-on scope

- Configurable PC-wide incremental index, SQLite search, date/year timeline grouping, smart collections, storage analysis, folder-size scans, duplicate candidates, Office/archive/audio expansion, content search, advanced Windows integration.

Definition of done

The app is ready for everyday personal use when it can browse and manage real files, show correct content previews for supported types, remain responsive on large folders, handle permission failures and disconnected drives, organize virtually without moving originals, and build/install/uninstall cleanly.

> Final recommendation
> Make the first release exceptional at three things: premium browsing, real previews, and safe file operations. Add full-PC indexing and storage intelligence only after those foundations are reliable.

End of KiroVault Desktop App Blueprint